

DSC 40B: THEORETICAL FOUNDATIONS OF DATA SCIENCE II

SORTING II

MERGE SORT IS OPTIMAL AND WE ARE STILL NOT DONE

SETTLED

Merge sort is $\Theta(n \log n)$ in the best case, the worst case, and the average case. No comparison sort has a better worst case.

OWED

It copies the array on every split, so it needs $\Theta(n)$ extra space. It also does the same work on sorted input as on random input.

PARTITION AND QUICKSORT

PARTITIONING

Pick any element of the array and call it the pivot. Move everything smaller than it to its left, everything larger to its right.

01 The pivot is now in its final position

Nothing to its left is larger and nothing to its right is smaller. That is what it means to be sorted. The pivot never moves again.

02 The two sides are independent problems

No element on the left will ever need to be compared with an element on the right. Each side is a smaller instance of the same problem.

03 Recurse on both sides, and stop at size one

A range of one element is already sorted. Every element eventually gets its turn as a pivot, so every element eventually lands.

WHAT YOU GET

p is in its final sorted position. It never moves again, no matter what the rest of the algorithm does.

WHAT YOU DO NOT GET

Neither side is sorted. Partition only guarantees the split, not the order inside it.

WHY THAT IS ENOUGH

Recurse on both sides. Every element eventually gets a turn as a pivot, and lands in its final position.

ONE PASS, TRACED

Partition arr = [7, 2, 9, 1, 5, 6] around the last element, pivot = 6. The barrier starts at index 0.

I	ARR[I]	TEST	ARRAY AFTER THE STEP	BARRIER
0	7	7 < 6 is false	7 2 9 5 6	0
1	2	2 < 6, swap with index 0	2 7 9 5 6	1
2	9	9 < 6 is false	2 7 9 5 6	1
3	1	1 < 6, swap with index 1	2 1 9 7 5 6	2
4	5	5 < 6, swap with index 2	2 1 5 7 9 6	3
end		final swap of pivot into place	2 1 5 6 9 7	3

Index 3 holds 6, and 6 is the fourth smallest element. It is done. The left side [2, 1, 5] and the right side [9, 7] are unsorted, and neither will ever need to look at the other again.

PARTITION THE ALGORITHM

```
def partition(arr, start, stop):  
    """Partition arr[start:stop] around arr[stop-1]."""  
    pivot = arr[stop - 1]  
    barrier_ix = start  
  
    for i in range(start, stop - 1):  
        if arr[i] < pivot:  
            arr[i] = arr[barrier_ix]  
            arr[barrier_ix] = arr[i]  
            barrier_ix += 1  
  
    arr[barrier_ix] = arr[stop - 1]  
    arr[stop - 1] = arr[barrier_ix]  
    return barrier_ix
```

LOOP INVARIANT

After the a th iteration, `arr[start:barrier_ix]` holds only elements less than pivot, and the a elements examined so far that are not less than pivot sit in `arr[barrier_ix:start+a]`.

WHAT THE BARRIER MEANS NOW

It separates smaller than pivot from not smaller than pivot.

THE FINAL SWAP

The loop skips the pivot itself. One swap at the end drops it into the gap at `barrier_ix`, which is exactly its sorted position.

QUICKSORT: PARTITION, RECURSE, STOP

```
def quicksort(arr, start=0, stop=None):  
    if stop is None:  
        stop = len(arr)  
  
    if stop - start <= 1:  
        return  
  
    pivot_ix = partition(arr, start, stop)  
    quicksort(arr, start, pivot_ix)  
    quicksort(arr, pivot_ix + 1, stop)
```

THE THREE CHECKS

Base case. A range of zero or one element is already sorted. The function returns without touching it.

Strictly smaller. The pivot is excluded from both calls, so together they cover $n - 1$ elements. Every call shrinks.

Combination. Assume both calls sort their ranges. The pivot is already correct and separates them, so the whole range is sorted.

THE PIVOT DECIDES THE RECURRENCE

Partition costs $\Theta(n)$ at every level. The only free variable is how the $n - 1$ remaining elements get divided between the two calls.

A BALANCED SPLIT

The pivot is the median. Each call gets about $n/2$ elements.

$$T(n) = 2 T(n/2) + n$$

This is the merge sort recurrence. Unrolling it gives $\Theta(n \log n)$.

A VERY BAD SPLIT

What if the pivot is the minimum or the maximum. One call gets $n - 1$ elements, the other gets zero.

$$T(n) = T(n - 1) + n$$

Unrolling gives $n + (n-1) + \dots + 1$, which is $\Theta(n^2)$.

THE WORST CASE IS THE INPUT YOU ALREADY SORTED

Take pivot = arr[stop-1] and feed it an ascending array. The last element is the maximum of every range, on every call.

CALL	RANGE	PIVOT	LEFT CALL	RIGHT CALL
1	[1, 2, 3, 4, 5]	5	4 elements	empty
2	[1, 2, 3, 4]	4	3 elements	empty
3	[1, 2, 3]	3	2 elements	empty
4	[1, 2]	2	1 element	empty

$$T(n) = T(n-1) + n = n + (n-1) + \dots + 2 + 1 = n(n+1)/2 = \Theta(n^2)$$

THINK BEFORE YOU PICK THE PIVOT

STRATEGY	WORST CASE	TRIGGERED BY	VERDICT
Last element	$\Theta(n^2)$	sorted or reverse sorted input	Never ship this.
Middle element	$\Theta(n^2)$	an input built to defeat it	Fine until someone constructs the bad case.
Median of three	$\Theta(n^2)$	a rarer constructed input	Cheap, and good on real data.
Uniform random	$\Theta(n^2)$	bad luck only, not any fixed input	Expected $\Theta(n \log n)$. This is the one to use.
True median	$\Theta(n \log n)$	nothing	Guaranteed, but the median costs more than it saves.

QUICKSELECT

Selection problem: given arr and an index k, return the element that would sit at arr[k] if arr were sorted.

```
def quickselect(arr, k, start=0, stop=None):
    if stop is None:
        stop = len(arr)

    pivot_ix = partition(arr, start, stop)

    if pivot_ix == k:
        return arr[k]
    elif k < pivot_ix:
        return quickselect(arr, k, start, pivot_ix)
    else:
        return quickselect(arr, k, pivot_ix + 1, stop)
```

ONE LINE OF DIFFERENCE

Quicksort recurses on both sides. Quickselect recurses on one. After partition returns, the pivot index tells you which side k is on, and the other side is discarded without ever being examined.

WHY IT IS CORRECT

Partition puts the pivot in its final sorted position. So pivot_ix is a sorted rank, and comparing it to k is comparing two positions in the same sorted order.

DISCARDING HALF THE WORK CHANGES THE SUM

QUICKSORT

Two calls per level.

$$T(n) = 2 T(n/2) + n$$

The n at every level is paid $\Theta(\log n)$ times. Total $\Theta(n \log n)$.

QUICKSELECT

One call per level.

$$T(n) = T(n/2) + n$$

The cost per level halves. $n + n/2 + n/4 + \dots < 2n$.
Total $\Theta(n)$.

QUICKSELECT, WORST CASE

A degenerate pivot every time.

$$T(n) = T(n - 1) + n$$

Still $\Theta(n^2)$. Randomize the pivot and the expected cost is $\Theta(n)$.

A geometric series with ratio less than 1 sums to a constant multiple of its first term. A series of equal terms does not.

MERGE SORT VS QUICKSORT

Both algorithms split, recurse, and combine. They disagree about which of those steps is allowed to be expensive.

MERGE SORT

Split at the midpoint. That choice ignores the data, so the halves are balanced but unordered relative to each other.

All ordering work happens in merge, on the way back up.

Cost: $\Theta(n)$ per level, $\Theta(\log n)$ levels.

QUICKSORT

Split around a pivot value. Everything smaller lands left, everything larger lands right, so the halves are already ordered relative to each other.

There is no combine step.

Cost: $\Theta(n)$ per level, and the number of levels depends on the pivot.

TWO $\Theta(N \log N)$ SORTS, DIFFERENT PROMISES

PROPERTY	MERGE SORT	QUICKSORT	WHY IT MATTERS
Worst case	$\Theta(n \log n)$	$\Theta(n^2)$	Merge sort is the one you pick under a deadline.
Average case	$\Theta(n \log n)$	$\Theta(n \log n)$	Quicksort wins on constants, not on growth rate.
Extra space	$\Theta(n)$	$\Theta(\log n)$ stack	Quicksort sorts in place. Merge sort cannot.
Stable	Yes	No	Partition swaps distant elements and reorders equal keys.
Adaptive	No	No	Neither one is faster on nearly sorted input.
Work happens in	Combine	Divide	merge builds the order. partition places it.